SE4041 – Mobile Application Design and Development

2025 – Tutorial 01 · Kotlin basics · Code and program output (Kotlin 2.0.21)

Introduction to basic syntax

Code (intro/Intro.kt)

```
package tutorial.intro

// Introduction to basic syntax
fun main(args: Array<String>) {
    println("Hello World")
    println(args.contentToString())
    print("Hi I'm ")
    println(42)
    println(3.141592)
}
```

Output

```
Hello World
[]
Hi I'm 42
3.141592
```

1. Variables

Code (variables/Variables.kt)

```
package tutorial.variables

// 1. Variables

// You can declare variables at the top level.
val pi = 3.14
var r = 0

fun main(args: Array<String>) {
    // Read-only local variables
    val a: Int = 1  // immediate assignment
    val b = 2       // `Int` type is inferred
    val c: Int      // Type required when no initializer is provided
    c = 3           // deferred assignment

    // Variables that can be re-assigned
    var x = 5       // `Int` type is inferred
    x += 1

    println("a = $a, b = $b, c = $c, x = $x")

    r = 7
    println("Circumference = " + 2 * pi * r)
}
```

Output

```
a = 1, b = 2, c = 3, x = 6
Circumference = 43.96
```

2. Functions

Code (functions/Functions.kt)

```
package tutorial.functions

// 2. Functions
val pi = 3.14
var r = 0

fun main(args: Array<String>) {
    r = 7
    println("Circumference = " + 2 * pi * r)
    println("Area is " + calculateArea(10.0))
}

fun calculateArea(r: Double): Double {
    return pi * r * r
}
```

Output

```
Circumference = 43.96
Area is 314.0
```

3. Conditional Expressions

Code (conditionals/Conditionals.kt)

```
package tutorial.conditionals

// 3. Conditional Expressions
var r = 0

fun main() {
    r = 7
    if (r > 5) {
        println("r is greater than 5")
    } else {
        println("r is less than 5")
    }
    // if as a single-line statement
    if (r < 5) print("a") else print("b")
    println()
}
```

Output

```
r is greater than 5
b
```

4. Iterations

Code (iterations/Iterations.kt)

```
package tutorial.iterations

// 4. Iterations
fun main() {
    val names = listOf("John", "Jane", "Tom", "Jack", "Anne", "Kane")
    var i = 0

    for (name in names) {
        println("Hello, $name")
    }

    for (index in names.indices) {
        println(index)
    }

    while (i < names.size) {
        println("name at $i is ${names[i]}")
        i++
    }
}
```

Output

```
Hello, John
Hello, Jane
Hello, Tom
Hello, Jack
Hello, Anne
Hello, Kane
0
1
2
3
4
5
name at 0 is John
name at 1 is Jane
name at 2 is Tom
name at 3 is Jack
name at 4 is Anne
name at 5 is Kane
```

5. When expression

Code (whenexpr/WhenExpression.kt)

```
package tutorial.whenexpr

// 5. When expression in Kotlin
fun main() {
    println(whatIs(1))
    println(whatIs(10))
    println(whatIs(9223372036854775800))
    println(whatIs("Hello"))
    println(whatIs(5.0))
}

fun whatIs(obj: Any): String =
    when (obj) {
        1 -> "one"
        "Hello" -> "Greetings"
        is Long -> "Long"
        !is String -> "not a string"
        else -> "Unknown"
    }
```

Output

```
one
not a string
Long
Greetings
not a string
```

6. Collections

Code (collections/Collections.kt)

```
package tutorial.collections

// 6. Collections
fun main() {
    // Lists
    val numbers = listOf(1, 2, 3, 4, 5)
    println("List: $numbers")

    // Sets
    val uniqueNumbers = setOf(1, 2, 3, 2, 4, 5)
    println("Set: $uniqueNumbers")

    // Maps
    val capitals = mapOf("USA" to "Washington D.C.", "UK" to "London", "India" to "New Delhi")
    println("Capitals: $capitals")
}
```

Output

```
List: [1, 2, 3, 4, 5]
Set: [1, 2, 3, 4, 5]
Capitals: {USA=Washington D.C., UK=London, India=New Delhi}
```

6.1 Lists

Code (collections/lists/Lists.kt)

```
package tutorial.collections.lists

// 6.1 Lists
fun main() {
    // Immutable list
    val numbers = listOf(1, 2, 3, 4, 5)
    println("Immutable List: $numbers")

    // Mutable list
    val mutableNumbers = mutableListOf(1, 2, 3, 4, 5)
    mutableNumbers.add(6)
    println("Mutable List: $mutableNumbers")
}
```

Output

```
Immutable List: [1, 2, 3, 4, 5]
Mutable List: [1, 2, 3, 4, 5, 6]
```

6.2 Sets

Code (collections/sets/Sets.kt)

```
package tutorial.collections.sets

// 6.2 Sets
fun main() {
    // Immutable set
    val uniqueNumbers = setOf(1, 2, 3, 2, 4, 5)
    println("Immutable Set: $uniqueNumbers")

    // Mutable set
    val mutableUniqueNumbers = mutableSetOf(1, 2, 3, 2, 4, 5)
    mutableUniqueNumbers.add(6)
    println("Mutable Set: $mutableUniqueNumbers")
}
```

Output

```
Immutable Set: [1, 2, 3, 4, 5]
Mutable Set: [1, 2, 3, 4, 5, 6]
```

6.3 Maps

Code (collections/maps/Maps.kt)

```
package tutorial.collections.maps

// 6.3 Maps
fun main() {
    // Immutable map
    val capitals = mapOf("USA" to "Washington D.C.", "UK" to "London", "India" to "New Delhi")
    println("Immutable Map: $capitals")

    // Mutable map
    val mutableCapitals = mutableMapOf("USA" to "Washington D.C.", "UK" to "London", "India" to "New
Delhi")
    mutableCapitals["Germany"] = "Berlin"
    println("Mutable Map: $mutableCapitals")
}
```

Output

```
Immutable Map: {USA=Washington D.C., UK=London, India=New Delhi}
Mutable Map: {USA=Washington D.C., UK=London, India=New Delhi, Germany=Berlin}
```

7.1 Filtering

Code (operations/filtering/Filtering.kt)

```
package tutorial.operations.filtering

// 7.1 Filtering
fun main() {
    val numbers = listOf(1, 2, 3, 4, 5)
    // Filter elements greater than 3
    val filteredNumbers = numbers.filter { it > 3 }
    println("Filtered Numbers: $filteredNumbers")
}
```

Output

```
Filtered Numbers: [4, 5]
```

7.2 Mapping

Code (operations/mapping/Mapping.kt)

```
package tutorial.operations.mapping

// 7.2 Mapping
fun main() {
    val numbers = listOf(1, 2, 3, 4, 5)
    // Square each element
    val squaredNumbers = numbers.map { it * it }
    println("Squared Numbers: $squaredNumbers")
}
```

Output

```
Squared Numbers: [1, 4, 9, 16, 25]
```

7.3 Sorting

Code (operations/sorting/Sorting.kt)

```
package tutorial.operations.sorting

// 7.3 Sorting
fun main() {
    val numbers = listOf(5, 2, 8, 1, 4)
    // Sort in ascending order
    val sortedNumbers = numbers.sorted()
    println("Sorted Numbers: $sortedNumbers")
    // Sort in descending order
    val descendingNumbers = numbers.sortedDescending()
    println("Descending Numbers: $descendingNumbers")
}
```

Output

```
Sorted Numbers: [1, 2, 4, 5, 8]
Descending Numbers: [8, 5, 4, 2, 1]
```

7.4 Grouping

Code (operations/grouping/Grouping.kt)

```
package tutorial.operations.grouping

// 7.4 Grouping
fun main() {
    val names = listOf("Alice", "Bob", "Charlie", "Anna")
    // Group names by their first letter
    val groupedNames = names.groupBy { it.first() }
    println("Grouped Names: $groupedNames")
}
```

Output

```
Grouped Names: {A=[Alice, Anna], B=[Bob], C=[Charlie]}
```

OOP 1. Creating classes and instances

Code (oop/classes/Classes.kt)

```
package tutorial.oop.classes

// OOP 1. Creating classes and instances
fun main() {
    val student = Person("John Smith")
}

class Person(name: String) {
    val name = "Name is $name".also(::println)

    init {
        println("First initializer block that prints the $name")
    }

    val nameSize = "name size is ${name.length}".also(::println)

    init {
        println("Second initializer block that prints the ${name.length}")
    }
}
```

Output

```
Name is John Smith
First initializer block that prints the John Smith
name size is 10
Second initializer block that prints the 10
```

OOP 2. Inheritance

Code (oop/inheritance/Inheritance.kt)

```
package tutorial.oop.inheritance

// OOP 2. Inheritance
fun main() {
    val square1 = Rectangle(4.0, 4.0)
    println(square1.perimeter)
}

open class Shape

class Rectangle(var length: Double, var width: Double) : Shape() {
    var perimeter: Double = (length + width) * 2
}
```

Output

```
16.0
```